

基于 Transformer 的倒立摆控制

系统建模、最优控制与深度学习控制的完整理论及编码实现

技术文档

2026 年 9 月 2 日

摘要

本文完整阐述用 **Transformer** 神经网络控制倒立摆 (Cart-Pole) 的理论与实现。内容覆盖四个层面：**(i) 系统建模**——从质点杆模型出发，经拉格朗日方程严格推导非线性动力学，再由一阶泰勒展开得到状态空间线性化模型，并分析开环稳定性与可控性；**(ii) 最优控制**——由庞特里亚金极小值原理完整导出线性二次型调节器 (LQR) 的 Riccati 方程，给出离散化迭代求解算法；**(iii) 深度学习**——系统讲解 Transformer 的缩放点积注意力、多头注意力、位置编码、前馈网络与残差连接各组件及其数学依据；**(iv) 编码实现**——以纯 NumPy 手写物理仿真、LQR 求解、Transformer 前向与反向传播、训练循环，并对每段代码逐行说明。实验表明，经模仿学习训练的 Transformer 控制器能稳定维持倒立摆平衡，输出与 LQR 专家高度一致。

目录

1 引言	3
2 倒立摆系统建模	3
2.1 系统描述、坐标系与假设	3
2.2 运动学：质心位置与速度	4
2.3 拉格朗日量：动能与势能	4
2.4 欧拉-拉格朗日方程	4
2.5 显式化：解出 $\ddot{\theta}$ 与 $\ddot{x}$	5
2.6 平衡点线性化：一阶泰勒展开	6
2.7 开环稳定性与可控性	7
3 LQR 最优控制	7
3.1 最优控制问题形式化	7
3.2 最优控制律推导：庞特里亚金极小值原理	8
3.3 离散化与离散 Riccati 迭代	8
3.4 权重选取与求解结果	9
3.5 作为专家策略	9

4 Transformer 理论基础	9
4.1 注意力机制的动机	9
4.2 缩放点积注意力	9
4.3 多头注意力	10
4.4 位置编码	10
4.5 前馈网络、残差与层归一化	10
5 Transformer 作为控制器	10
5.1 控制问题的序列化	10
5.2 模仿学习 (Behavior Cloning)	11
6 网络架构与训练	11
6.1 网络配置	11
6.2 前向传播	11
6.3 训练配置	12
7 编码实现详解	12
7.1 物理仿真：非线性动力学与 RK4 积分	12
7.2 LQR 求解：线性化与离散 Riccati 迭代	13
7.3 位置编码与嵌入	14
7.4 多头注意力：前向	14
7.5 多头注意力：反向	14
7.6 LayerNorm：前向与反向	15
7.7 前馈网络与整体前向	15
7.8 数据收集与训练循环	16
7.9 梯度检查	16
8 实验结果	17
8.1 训练收敛	17
8.2 闭环稳定性	17
8.3 扰动鲁棒性	17
8.4 与 LQR 对比与注意力可视化	17
9 手动训练与运行指南	17
9.1 环境准备	18
9.2 文件清单	18
9.3 一键运行与脚本内部执行顺序	18
9.4 分步手动执行 (可选)	19
9.5 超参数调整	19
9.6 常见问题	19
10 结论	20

1 引言

倒立摆系统由可沿水平轨道运动的小车与铰接于小车顶端的摆杆组成，控制目标为施加水平力 F ，使摆杆维持在竖直向上的**不稳定平衡点**附近，同时小车位置回归原点。该系统集**欠驱动**（仅一个控制输入、两个自由度）、**非线性**、**开环不稳定**于一身，是控制理论与机器人学的经典验证平台。

控制方法大致沿两条路线演进：

- **基于模型**：PID、LQR、模型预测控制（MPC）。需要建立并线性化模型，控制器结构明确、可解释，但对建模精度敏感。
- **数据驱动**：强化学习、模仿学习。直接从数据学习策略，无需精确模型，但可解释性较弱。

Transformer [1] 凭借**多头自注意力**在序列建模上取得巨大成功。其本质能力——对序列任意位置间的长程依赖直接建模——与控制问题高度契合：控制器本质上是由**观测历史到当前动作**的序列决策器。本文将倒立摆控制形式化为序列决策任务，用 Transformer 作为策略网络，经模仿学习训练，并给出从建模到编码的完整闭环。

2 倒立摆系统建模

2.1 系统描述、坐标系与假设

图 1 为系统示意。设小车质量 M ，摆杆质量 m ，摆杆半长（质心到铰点距离） l ，重力加速度 g 。广义坐标取小车水平位移 x （向右为正）与摆杆偏角 θ （相对竖直向上，顺时针为正）。控制输入为水平力 F 。

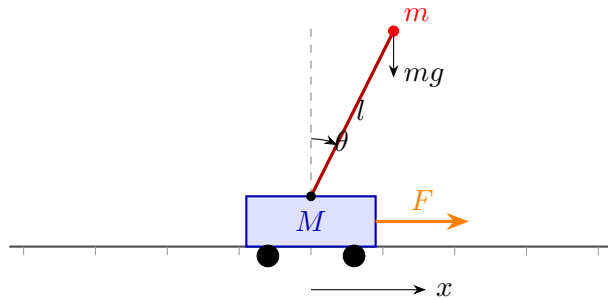


图 1: 倒立摆（Cart-Pole）系统示意图：小车质量 M 、摆杆质量 m （质点模型）、半长 l 、偏角 θ （相对竖直向上）、水平控制力 F 。

关键假设（质点杆模型）：将摆杆质量 m 视为集中于距铰点 l 处的单个质点，忽略杆的转动惯量。该假设使非线性动力学与后续线性化结果**严格自治**——若采用均匀杆（绕质心惯量 $ml^2/3$ ），线性化结果将含有 $\frac{4}{3}$ 因子，而本文的质点模型可精确退化为教科书标准形式。参数取值见表 1。

表 1: 系统参数与状态变量

符号	含义	取值
M	小车质量	1.0 kg
m	摆杆质量	0.1 kg
l	摆杆半长	0.5 m
g	重力加速度	9.81 m/s ²
x	小车位置	状态量
θ	摆杆偏角	状态量
F	水平控制力	控制输入

2.2 运动学：质心位置与速度

摆杆质心在惯性系中的坐标为

$$x_{\text{cm}} = x + l \sin \theta, \quad y_{\text{cm}} = l \cos \theta. \quad (1)$$

对时间求导得质心速度：

$$\dot{x}_{\text{cm}} = \dot{x} + l\dot{\theta} \cos \theta, \quad \dot{y}_{\text{cm}} = -l\dot{\theta} \sin \theta. \quad (2)$$

2.3 拉格朗日量：动能与势能

系统动能 T 由小车平动与杆质心运动两部分构成：

$$\begin{aligned} T &= \underbrace{\frac{1}{2}M\dot{x}^2}_{\text{小车平动}} + \underbrace{\frac{1}{2}m(\dot{x}_{\text{cm}}^2 + \dot{y}_{\text{cm}}^2)}_{\text{杆质心运动}} \\ &= \frac{1}{2}M\dot{x}^2 + \frac{1}{2}m \left[(\dot{x} + l\dot{\theta} \cos \theta)^2 + (l\dot{\theta} \sin \theta)^2 \right]. \end{aligned} \quad (3)$$

展开平方项并利用 $\cos^2 \theta + \sin^2 \theta = 1$ ：

$$\begin{aligned} T &= \frac{1}{2}M\dot{x}^2 + \frac{1}{2}m(\dot{x}^2 + 2l\dot{x}\dot{\theta} \cos \theta + l^2\dot{\theta}^2 \cos^2 \theta + l^2\dot{\theta}^2 \sin^2 \theta) \\ &= \frac{1}{2}(M+m)\dot{x}^2 + ml\dot{x}\dot{\theta} \cos \theta + \frac{1}{2}ml^2\dot{\theta}^2. \end{aligned} \quad (4)$$

以摆杆竖直向下为零势能点，势能为

$$V = mg y_{\text{cm}} = mgl \cos \theta. \quad (5)$$

拉格朗日量

$$\mathcal{L} = T - V = \frac{1}{2}(M+m)\dot{x}^2 + ml\dot{x}\dot{\theta} \cos \theta + \frac{1}{2}ml^2\dot{\theta}^2 - mgl \cos \theta. \quad (6)$$

2.4 欧拉-拉格朗日方程

对广义坐标 $q \in \{x, \theta\}$ ，欧拉-拉格朗日方程为

$$\frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{q}} \right) - \frac{\partial \mathcal{L}}{\partial q} = Q_q, \quad (7)$$

其中广义力 $Q_x = F$ （外力沿 x 方向做功）， $Q_\theta = 0$ （摆杆无外力矩）。

对 x 方向。 计算各偏导：

$$\frac{\partial \mathcal{L}}{\partial \dot{x}} = (M + m)\dot{x} + ml\dot{\theta} \cos \theta, \quad (8)$$

$$\frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{x}} \right) = (M + m)\ddot{x} + ml\ddot{\theta} \cos \theta - ml\dot{\theta}^2 \sin \theta, \quad (9)$$

$$\frac{\partial \mathcal{L}}{\partial x} = 0. \quad (10)$$

代入得第一个运动方程

$$(M + m)\ddot{x} + ml\ddot{\theta} \cos \theta - ml\dot{\theta}^2 \sin \theta = F \quad (11)$$

对 θ 方向。 同理：

$$\frac{\partial \mathcal{L}}{\partial \dot{\theta}} = ml\dot{x} \cos \theta + ml^2\dot{\theta}, \quad (12)$$

$$\frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{\theta}} \right) = ml\ddot{x} \cos \theta - ml\dot{x}\dot{\theta} \sin \theta + ml^2\ddot{\theta}, \quad (13)$$

$$\frac{\partial \mathcal{L}}{\partial \theta} = -ml\dot{x}\dot{\theta} \sin \theta + mgl \sin \theta. \quad (14)$$

代入得第二个运动方程，注意到 $-ml\dot{x}\dot{\theta} \sin \theta$ 两项相消：

$$ml\ddot{x} \cos \theta + ml^2\ddot{\theta} - mgl \sin \theta = 0. \quad (15)$$

两边同除以 ml ：

$$\ddot{x} \cos \theta + l\ddot{\theta} - g \sin \theta = 0 \quad (16)$$

2.5 显式化：解出 $\ddot{\theta}$ 与 $\ddot{x}$

式 (11)、(16) 为关于 $\ddot{x}, \ddot{\theta}$ 的线性方程组。由式 (11) 解出 $\ddot{x}$ ：

$$\ddot{x} = \frac{F - ml\ddot{\theta} \cos \theta + ml\dot{\theta}^2 \sin \theta}{M + m}. \quad (17)$$

代入式 (16) 消去 $\ddot{x}$ ：

$$\frac{F \cos \theta + ml\dot{\theta}^2 \sin \theta \cos \theta}{M + m} - \frac{ml \cos^2 \theta}{M + m} \ddot{\theta} + l\ddot{\theta} = g \sin \theta. \quad (18)$$

合并 $\ddot{\theta}$ 项：

$$\ddot{\theta} \left[l - \frac{ml \cos^2 \theta}{M + m} \right] = g \sin \theta - \frac{\cos \theta (F + ml\dot{\theta}^2 \sin \theta)}{M + m}. \quad (19)$$

最终得到

$$\ddot{\theta} = \frac{g \sin \theta - \cos \theta \frac{F + ml\dot{\theta}^2 \sin \theta}{M + m}}{l \left(1 - \frac{m \cos^2 \theta}{M + m} \right)} \quad (20)$$

回代得

$$\ddot{x} = \frac{F + ml (\dot{\theta}^2 \sin \theta - \ddot{\theta} \cos \theta)}{M + m} \quad (21)$$

式 (20)、(21) 即仿真所用的完整非线性动力学。定义状态向量 $\mathbf{z} = [x, \dot{x}, \theta, \dot{\theta}]^\top$ 与输入 $u = F$, 得状态方程

$$\dot{\mathbf{z}} = f(\mathbf{z}, u) = \begin{bmatrix} \dot{x} \\ \ddot{x} \\ \dot{\theta} \\ \ddot{\theta} \end{bmatrix}. \quad (22)$$

2.6 平衡点线性化：一阶泰勒展开

平衡点。 直立平衡条件为 $\theta = 0, \dot{\theta} = 0, \dot{x} = 0, u = 0$ 。在平衡点邻域做一阶泰勒展开：

$$\dot{\mathbf{z}} \approx f(\mathbf{z}^*, u^*) + \underbrace{\frac{\partial f}{\partial \mathbf{z}} \Big|_*}_{\mathbf{A}} (\mathbf{z} - \mathbf{z}^*) + \underbrace{\frac{\partial f}{\partial u} \Big|_*}_{\mathbf{B}} u. \quad (23)$$

求 $\mathbf{A}$ 与 $\mathbf{B}$ 。 状态方程中, $f_1 = \dot{x}, f_3 = \dot{\theta}$ 为恒等式, 故 $\mathbf{A}$ 的第 1、3 行分别为 $[0, 1, 0, 0], [0, 0, 0, 1]$ 。关键在于 $f_2 = \ddot{x}$ 与 $f_4 = \ddot{\theta}$ 的偏导。

先求 $\ddot{\theta}$ 对 θ 与 u 的偏导。记式 (20) 的分子为 $N(\theta, u)$ 、分母为 $D(\theta)$ ：

$$N = g \sin \theta - \frac{\cos \theta (u + ml \dot{\theta}^2 \sin \theta)}{M + m}, \quad D = l \left(1 - \frac{m \cos^2 \theta}{M + m} \right). \quad (24)$$

在平衡点 ($\theta = \dot{\theta} = u = 0$) 处：

$$\frac{\partial N}{\partial \theta} \Big|_* = g \cos \theta = g, \quad \frac{\partial D}{\partial \theta} \Big|_* = 0, \quad D|_* = l \left(1 - \frac{m}{M + m} \right) = \frac{lM}{M + m}. \quad (25)$$

由商的求导法则 $\partial \ddot{\theta} / \partial \theta = (N_\theta D - N D_\theta) / D^2$ ：

$$\frac{\partial \ddot{\theta}}{\partial \theta} \Big|_* = \frac{g \cdot D|_*}{D|_*^2} = \frac{g}{D|_*} = \frac{(M + m)g}{Ml}. \quad (26)$$

同理对 u ：

$$\frac{\partial N}{\partial u} = -\frac{\cos \theta}{M + m} \Big|_* = -\frac{1}{M + m}, \quad \frac{\partial \ddot{\theta}}{\partial u} \Big|_* = -\frac{1}{(M + m)} \cdot \frac{1}{D|_*} = -\frac{1}{Ml}. \quad (27)$$

对 $\dot{x}, \dot{\theta}$ 的偏导在平衡点均为零。故

$$\ddot{\theta} \approx \frac{(M + m)g}{Ml} \theta - \frac{1}{Ml} u. \quad (28)$$

关键： $\ddot{x}$ 的线性化须代入线性化后的 $\ddot{\theta}$ 。由式 (21)，

$$\ddot{x} = \frac{u + ml(\dot{\theta}^2 \sin \theta - \ddot{\theta} \cos \theta)}{M + m}. \quad (29)$$

在平衡点, $\dot{\theta}^2 \sin \theta$ 项对 $\theta, \dot{\theta}$ 的偏导均为零, 但 $\ddot{\theta} \cos \theta$ 项同时依赖 θ 与 u (因 $\ddot{\theta}$ 本身含 u)。代入式 (28) 得

$$\begin{aligned} \ddot{x} &\approx \frac{u - ml\ddot{\theta}}{M + m} = \frac{u - ml\left(\frac{(M+m)g}{Ml}\theta - \frac{u}{Ml}\right)}{M + m} \\ &= \frac{u - \frac{m(M+m)g}{M}\theta + \frac{m}{M}u}{M + m} = \frac{u\left(1 + \frac{m}{M}\right) - \frac{m(M+m)g}{M}\theta}{M + m} \\ &= \frac{u\frac{M+m}{M} - \frac{m(M+m)g}{M}\theta}{M + m} = -\frac{mg}{M}\theta + \frac{1}{M}u. \end{aligned} \quad (30)$$

此即教科书标准形式

$$\ddot{x} \approx -\frac{mg}{M}\theta + \frac{1}{M}u. \quad (31)$$

线性状态空间模型。 由式 (28)、(31) 得

$$\dot{\mathbf{z}} = \mathbf{A}\mathbf{z} + \mathbf{B}u, \quad (32)$$

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & -\frac{mg}{M} & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & \frac{(M+m)g}{Ml} & 0 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 0 \\ \frac{1}{M} \\ 0 \\ -\frac{1}{Ml} \end{bmatrix}. \quad (33)$$

2.7 开环稳定性与可控性

开环不稳定。 矩阵 $\mathbf{A}$ 的特征方程为 $\lambda^2(\lambda^2 - \omega^2) = 0$, 其中

$$\omega = \sqrt{\frac{(M+m)g}{Ml}} \approx 4.65 \text{ rad/s}. \quad (34)$$

特征值为 $\{0, 0, +\omega, -\omega\}$, 正实部特征值 $+\omega$ 表明开环不稳定——这正是需要反馈控制的原因。

完全可控。 构造可控性矩阵

$$\mathcal{C} = [\mathbf{B}, \mathbf{AB}, \mathbf{A}^2\mathbf{B}, \mathbf{A}^3\mathbf{B}]. \quad (35)$$

逐项计算:

$$\mathbf{AB} = \begin{bmatrix} 1/M \\ 0 \\ -1/(Ml) \\ 0 \end{bmatrix}, \quad \mathbf{A}^2\mathbf{B} = \begin{bmatrix} 0 \\ mg/(M^2l) \\ 0 \\ -(M+m)g/(M^2l^2) \end{bmatrix}, \quad (36)$$

$$\mathbf{A}^3\mathbf{B} = \begin{bmatrix} mg/(M^2l) \\ 0 \\ -(M+m)g/(M^2l^2) \\ 0 \end{bmatrix}. \quad (37)$$

该矩阵呈块结构, 其行列式可解析求出:

$$\det(\mathcal{C}) = \frac{g^2}{M^4l^4} > 0. \quad (38)$$

$\det(\mathcal{C}) \neq 0$ 即 $\text{rank}(\mathcal{C}) = 4$, 系统**完全可控**, 保证 LQR 最优控制存在且闭环可稳定。

3 LQR 最优控制

3.1 最优控制问题形式化

考虑线性时不变系统 $\dot{\mathbf{z}} = \mathbf{A}\mathbf{z} + \mathbf{B}u$. LQR 求使二次型代价最小化的状态反馈:

$$J = \int_0^\infty (\mathbf{z}^\top \mathbf{Q} \mathbf{z} + u^\top \mathbf{R} u) dt, \quad (39)$$

其中 $\mathbf{Q} = \mathbf{Q}^\top \succeq 0$ 为状态惩罚， $\mathbf{R} = \mathbf{R}^\top \succ 0$ 为控制惩罚。

代价项的物理含义： $\mathbf{z}^\top \mathbf{Q} \mathbf{z}$ 惩罚状态偏离零（平衡）的程度——对角元越大，对应状态被拉回零越“急”； $u^\top \mathbf{R} u$ 惩罚控制能量—— $\mathbf{R}$ 越大，控制越“省力”但响应越慢。二者构成性能与控制能耗的权衡。

3.2 最优控制律推导：庞特里亚金极小值原理

构造哈密顿函数（引入协态向量 λ ）：

$$\mathcal{H} = \mathbf{z}^\top \mathbf{Q} \mathbf{z} + u^\top \mathbf{R} u + \lambda^\top (\mathbf{A} \mathbf{z} + \mathbf{B} u). \quad (40)$$

协态方程（对状态求偏导取负）：

$$\dot{\lambda} = -\frac{\partial \mathcal{H}}{\partial \mathbf{z}} = -(2\mathbf{Q} \mathbf{z} + \mathbf{A}^\top \lambda). \quad (41)$$

最优性条件（对控制求偏导为零）：

$$\frac{\partial \mathcal{H}}{\partial u} = 2\mathbf{R} u + \mathbf{B}^\top \lambda = 0 \implies u = -\frac{1}{2} \mathbf{R}^{-1} \mathbf{B}^\top \lambda. \quad (42)$$

令协态为状态的线性函数 $\lambda = 2\mathbf{P} \mathbf{z}$ （ $\mathbf{P}$ 待求对称矩阵），代入式 (42)：

$$u = -\mathbf{R}^{-1} \mathbf{B}^\top \mathbf{P} \mathbf{z} = -\mathbf{K} \mathbf{z}, \quad \mathbf{K} = \mathbf{R}^{-1} \mathbf{B}^\top \mathbf{P}. \quad (43)$$

对 $\lambda = 2\mathbf{P} \mathbf{z}$ 求导，并代入闭环 $\dot{\mathbf{z}} = (\mathbf{A} - \mathbf{B} \mathbf{K}) \mathbf{z}$ ：

$$\dot{\lambda} = 2\mathbf{P} \dot{\mathbf{z}} = 2\mathbf{P}(\mathbf{A} - \mathbf{B} \mathbf{K}) \mathbf{z}. \quad (44)$$

与协态方程 (41) 联立（稳态时 $\dot{\mathbf{P}} = 0$ ）：

$$2\mathbf{P}(\mathbf{A} - \mathbf{B} \mathbf{K}) \mathbf{z} = -2(\mathbf{Q} + \mathbf{A}^\top \mathbf{P}) \mathbf{z}, \quad (45)$$

消去 $\mathbf{z}$ 并展开 $\mathbf{K}$ ，得到连续代数 Riccati 方程 (CARE)：

$$\boxed{\mathbf{A}^\top \mathbf{P} + \mathbf{P} \mathbf{A} - \mathbf{P} \mathbf{B} \mathbf{R}^{-1} \mathbf{B}^\top \mathbf{P} + \mathbf{Q} = 0} \quad (46)$$

解出 $\mathbf{P}$ 后由式 (43) 得最优增益 $\mathbf{K}$ 。

3.3 离散化与离散 Riccati 迭代

CARE (46) 的解析求解需 Schur 分解，工程上更常用离散化后迭代。取采样周期 Δt ，欧拉离散化：

$$\mathbf{A}_d = \mathbf{I} + \mathbf{A} \Delta t, \quad \mathbf{B}_d = \mathbf{B} \Delta t. \quad (47)$$

离散代价 $J_d = \sum_k (\mathbf{z}_k^\top \mathbf{Q} \mathbf{z}_k + u_k^\top \mathbf{R} u_k)$ 对应的离散 Riccati 方程为

$$\mathbf{P} = \mathbf{Q} + \mathbf{A}_d^\top \mathbf{P} \mathbf{A}_d - \mathbf{A}_d^\top \mathbf{P} \mathbf{B}_d (\mathbf{R} + \mathbf{B}_d^\top \mathbf{P} \mathbf{B}_d)^{-1} \mathbf{B}_d^\top \mathbf{P} \mathbf{A}_d. \quad (48)$$

据此迭代：

$$\mathbf{K} \leftarrow (\mathbf{R} + \mathbf{B}_d^\top \mathbf{P} \mathbf{B}_d)^{-1} \mathbf{B}_d^\top \mathbf{P} \mathbf{A}_d, \quad (49)$$

$$\mathbf{P} \leftarrow \mathbf{Q} + \mathbf{A}_d^\top \mathbf{P} \mathbf{A}_d - \mathbf{A}_d^\top \mathbf{P} \mathbf{B}_d \mathbf{K}. \quad (50)$$

当 $\Delta t \rightarrow 0$ 时离散解收敛到连续解。该迭代全局收敛于唯一的镇定解（可控性与可观测性保证）。

3.4 权重选取与求解结果

权重选取原则：希望角度 θ 尽快归零，故 $\mathbf{Q}$ 中 θ 的权重远大于其他；控制能耗次之。取

$$\mathbf{Q} = \text{diag}(1, 0.1, 50, 0.1), \quad R = 0.1. \quad (51)$$

$\Delta t = 0.01$ s，迭代约十余次即收敛，得

$$\mathbf{K} \approx \begin{bmatrix} -2.954 & -4.735 & -46.339 & -9.163 \end{bmatrix}. \quad (52)$$

$|K_3| = 46.339$ 远大于其余元素，说明控制强依赖角度，符合直觉； $K_4 = -9.163$ 提供角速度阻尼。

3.5 作为专家策略

LQR 在本文中既是**性能基准**，也是模仿学习的**专家教师**：

$$u_{\text{expert}}(\mathbf{z}) = -\mathbf{K}\mathbf{z}. \quad (53)$$

4 Transformer 理论基础

4.1 注意力机制的动机

注意力机制源于**键-值查询** (key-value query) 的检索类比：给定一个查询 $\mathbf{q}$ ，在”键值对”库中按”查询与键的相似度”检索信息，并对检索到的值做加权聚合。对序列建模而言，每个位置既是查询又是键值，于是得到**自注意力**——每个位置都”审视”整条序列，按相关性聚合其他位置的信息。

4.2 缩放点积注意力

给定查询 $\mathbf{Q}$ 、键 $\mathbf{K}$ 、值 $\mathbf{V}$ ，注意力为

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}, \quad (54)$$

其中 d_k 为键的维度。

为什么除以 $\sqrt{d_k}$ ？ 设查询与键的各分量独立同分布、均值为零、方差为 1，则内积 $\mathbf{q} \cdot \mathbf{k} = \sum_i q_i k_i$ 的均值为零、方差为

$$\text{Var}(\mathbf{q} \cdot \mathbf{k}) = \sum_{i=1}^{d_k} \text{Var}(q_i k_i) = d_k. \quad (55)$$

即内积的量级为 $\sqrt{d_k}$ 。若不缩放，随着 d_k 增大，内积落入 softmax 的饱和区，导致梯度消失；除以 $\sqrt{d_k}$ 使方差回到 1，保证 softmax 处于非饱和区、梯度稳定。

softmax 的作用：将注意力分数转化为非负、和为 1 的权重分布，实现对各位置信息的”软”选择（可微的加权）。

4.3 多头注意力

单头注意力只在一个子空间内计算相关性。多头注意力将 $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ 分别投影到 h 个子空间，并行计算 h 次注意力后拼接：

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O, \quad (56)$$

$$\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V). \quad (57)$$

动机：不同头在不同子空间学习不同关系模式（如一个头关注角度、另一个关注速度），等价于卷积的多通道，提升表达能力。本文取 $h = 4$ 、每头 $d_k = 8$ 。

4.4 位置编码

Transformer 无循环/卷积结构，序列顺序信息须显式注入。本文采用正弦/余弦位置编码：

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right), \quad (58)$$

$$\text{PE}(\text{pos}, 2i + 1) = \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right). \quad (59)$$

设计动机：不同频率的正弦波使不同维度的编码随位置以不同速率变化，允许模型通过线性组合学到相对位置关系；且可外推到训练时未见过的更长序列。

4.5 前馈网络、残差与层归一化

注意力子层后接两层前馈网络（FFN），中间维通常取 4 倍：

$$\text{FFN}(\mathbf{x}) = \text{ReLU}(\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2. \quad (60)$$

每个子层采用**残差连接与层归一化**（LayerNorm）：

$$\mathbf{x}' = \text{LayerNorm}(\mathbf{x} + \text{Sublayer}(\mathbf{x})). \quad (61)$$

残差连接缓解深层网络的梯度消失；层归一化对每个位置的特征维标准化（而非跨 batch），稳定训练、加速收敛：

$$\mu = \frac{1}{d} \sum_i x_i, \quad \sigma^2 = \frac{1}{d} \sum_i (x_i - \mu)^2, \quad \hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}, \quad y_i = \gamma_i \hat{x}_i + \beta_i. \quad (62)$$

5 Transformer 作为控制器

5.1 控制问题的序列化

控制器可抽象为“观测历史 $\rightarrow$ 动作”的映射。取 t 时刻最近 T 步状态构成输入序列

$$\mathbf{X}_t = [\mathbf{z}_{t-T+1}, \mathbf{z}_{t-T+2}, \dots, \mathbf{z}_t]^\top \in \mathbb{R}^{T \times 4}. \quad (63)$$

策略网络输出控制力

$$u_t = \pi_\phi(\mathbf{X}_t). \quad (64)$$

与 LQR 仅利用当前状态 $u = -\mathbf{K}\mathbf{z}_t$ 不同，Transformer 能利用**整段历史**并通过注意力**自适应加权**各历史步的贡献。控制闭环如图 2 所示。

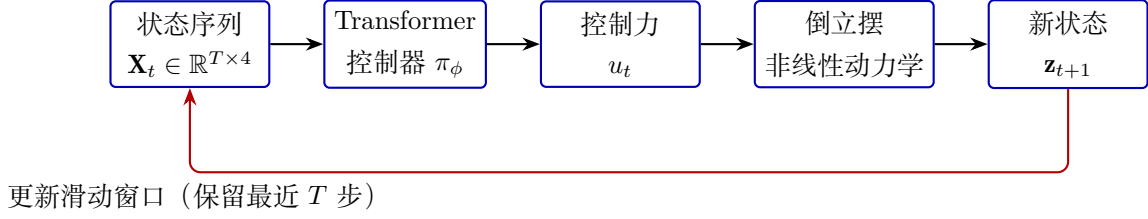


图 2: Transformer 控制闭环：状态序列经策略网络输出控制力，作用于倒立摆动力学，新状态滑入历史窗口构成下一时刻输入。

5.2 模仿学习 (Behavior Cloning)

训练目标为最小化策略输出与专家动作的均方误差：

$$\mathcal{L}(\phi) = \frac{1}{N} \sum_{i=1}^N (\pi_{\phi}(\mathbf{X}_i) - u_{\text{expert}}(\mathbf{z}_i))^2. \quad (65)$$

数据收集：以 LQR 专家驱动仿真，随机初始化状态，并周期性注入扰动，每个时间步采集样本 $(\mathbf{X}_t, u_{\text{expert}}(\mathbf{z}_t))$ 。扰动的加入使数据覆盖偏离平衡的状态，让策略学会**恢复能力**，缓解模仿学习固有的误差累积 (covariate shift) 问题。

6 网络架构与训练

6.1 网络配置

本文采用单层 Transformer 编码器块，配置见表 2。

表 2: Transformer 控制器网络配置

超参数	取值
序列长度 T	8
输入维度	4 ($[x, \dot{x}, \theta, \dot{\theta}]$)
嵌入维度 d_{model}	32
注意力头数 h	4 (每头 $d_k = 8$)
前馈网络维度 d_{ff}	64
编码器块数	1
输出维度	1 (控制力 u)

6.2 前向传播

图 3 给出了 Transformer 编码器块的完整前向结构（维度与残差连接标注于侧边）。

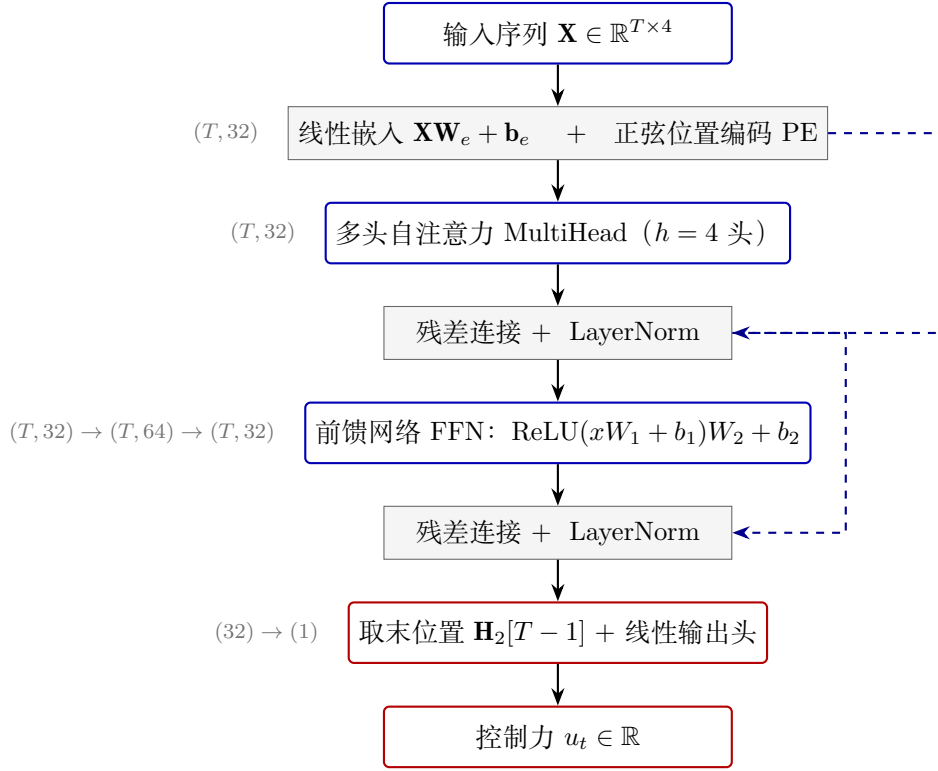


图 3: Transformer 控制器前向结构：输入序列经嵌入与位置编码后，依次经过「多头注意力 → 残差 + LayerNorm → 前馈网络 → 残差 + LayerNorm」，最后取末位置经线性头输出控制力。虚线为残差跳过连接。

记输入 $\mathbf{X} \in \mathbb{R}^{T \times 4}$ ，前向为

$$\mathbf{H} = \mathbf{X}\mathbf{W}_e + \mathbf{b}_e + \text{PE}, \quad (66)$$

$$\mathbf{H}_1 = \text{LayerNorm}(\mathbf{H} + \text{MultiHead}(\mathbf{H})), \quad (67)$$

$$\mathbf{H}_2 = \text{LayerNorm}(\mathbf{H}_1 + \text{FFN}(\mathbf{H}_1)), \quad (68)$$

$$u = \mathbf{H}_2[T-1] \mathbf{w}_o + b_o, \quad (69)$$

其中 $\mathbf{H}_2[T-1]$ 取输出序列的最后一个时间步（对应当前时刻），经线性输出头映射为标量控制力。

6.3 训练配置

损失即式 (65)。优化器 Adam（学习率 2×10^{-3} ），小批量 64，训练 60 epoch，数据约 5.8×10^4 条。反向传播为手写解析梯度（见第 7 节），经数值梯度检查验证（最大相对误差 $< 10^{-9}$ ）。

7 编码实现详解

实现仅依赖 NumPy，完整展示 Transformer 底层计算。所有代码与可运行脚本一致。

7.1 物理仿真：非线性动力学与 RK4 积分

```

1 def derivatives(z, u):
2     # z = [x, xd, th, thd]; 对应式(1)(2)
3     x, xd, th, thd = z
4     s, c = np.sin(th), np.cos(th)
5     denom = M + m
6     # eq(1): angular acceleration
7     thdd = (G*s - c*(u + m*L*thd*thd*s)/denom) / (L*(1 - m*c*c/denom))
8     # eq(2): cart acceleration
9     xdd = (u + m*L*(thd*thd*s - thdd*c)) / denom
10    return np.array([xd, xdd, thd, thdd])
11
12 def rk4(z, u, dt):
13     # 4th-order Runge-Kutta
14     k1 = derivatives(z, u)
15     k2 = derivatives(z + k1*dt/2, u)
16     k3 = derivatives(z + k2*dt/2, u)
17     k4 = derivatives(z + k3*dt, u)
18    return z + (k1 + 2*k2 + 2*k3 + k4)*dt/6

```

说明: derivatives 直接实现式 (20)、(21); rk4 为经典四阶龙格-库塔, 每步四次求导后加权平均, 四阶精度远优于欧拉法。

7.2 LQR 求解: 线性化与离散 Riccati 迭代

```

1 def build_AB():
2     A = np.array([[0, 1, 0, 0],
3                   [0, 0, -m*G/M, 0],
4                   [0, 0, 0, 1],
5                   [0, 0, (M+m)*G/(M*L), 0]])
6     B = np.array([[0], [1/M], [0], [-1/(M*L)]])
7     return A, B
8
9 def solve_lqr(Q_diag, R):
10    A, B = build_AB()
11    Q = np.diag(Q_diag)
12    Ad = np.eye(4) + A * DT          # Euler discretization
13    Bd = B * DT
14    P = Q.copy()
15    for _ in range(2000):
16        S = R + Bd.T @ P @ Bd        # denominator of eq(dlqr_k)
17        K = (Bd.T @ P @ Ad) / S
18        Pn = Q + Ad.T @ P @ Ad - Ad.T @ P @ Bd @ K  # eq(dlqr_p)
19        if np.abs(Pn - P).max() < 1e-13: break
20        P = Pn
21    return (Bd.T @ P @ Ad) / (R + Bd.T @ P @ Bd)    # 1x4 gain

```

说明: build_AB 实现式 (33); solve_lqr 直接实现式 (49)、(50) 的迭代, 收敛阈值 10^{-13} 。由于控制为标量, $\mathbf{R} + \mathbf{B}_d^T \mathbf{P} \mathbf{B}_d$ 是 1×1 标量, 求逆退化为除法。

7.3 位置编码与嵌入

```
1 def positional_encoding(T, d):
2     pe = np.zeros((T, d))
3     for t in range(T):
4         for i in range(0, d, 2):
5             pe[t, i] = np.sin(t / 10000**(i/d))
6             pe[t, i+1] = np.cos(t / 10000**(i/d))
7     return pe
8 PE = positional_encoding(T, D_MODEL) # (T, 32)
```

说明: 实现正弦/余弦位置编码, t 为位置、 i 为偶数维度索引。嵌入层为线性变换 $\mathbf{H} = \mathbf{X}\mathbf{W}_e + \mathbf{b}_e$ (参数 $\mathbf{W}_e \in \mathbb{R}^{4 \times 32}$)。

7.4 多头注意力: 前向

```
1 def attention_forward(h, p):
2     B, T, d = h.shape
3     Q = h @ p['Wq'] + p['bq'] # (B,T,d)
4     K = h @ p['Wk'] + p['bk']
5     V = h @ p['Wv'] + p['bv']
6     # split heads: (B,T,d) -> (B,H,T,DK)
7     Qh = Q.reshape(B, T, H, DK).transpose(0, 2, 1, 3)
8     Kh = K.reshape(B, T, H, DK).transpose(0, 2, 1, 3)
9     Vh = V.reshape(B, T, H, DK).transpose(0, 2, 1, 3)
10    # eq(attention): scaled dot-product + softmax
11    scores = Qh @ Kh.transpose(0, 1, 3, 2) / np.sqrt(DK) # (B,H,T,T)
12    attn = softmax(scores, axis=-1) # normalize along key dim
13    ctx = (attn @ Vh).transpose(0, 2, 1, 3).reshape(B, T, d)
14    out = ctx @ p['Wo'] + p['bo']
15    return out, (h, Qh, Kh, Vh, scores, attn, ctx)
```

维度追踪: h 为 $(B, T, 32)$; 投影后 $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ 均为 $(B, T, 32)$; $\text{reshape}(B, T, H, DK).transpose(0, 2, 1, 3)$ 将 32 维按 4 头切分为 $(B, 4, T, 8)$, 其中 $DK = d/H = 8$ 。scores 为 $(B, 4, T, T)$, 第 $[q, k]$ 元为查询位置 q 与键位置 k 的缩放内积。

7.5 多头注意力: 反向

```
1 def attention_backward(dout, cache, p):
2     h, Qh, Kh, Vh, scores, attn, ctx, Q, K, V = cache
3     B, T, d = h.shape
4     dctx = dout @ p['Wo'].T # (B,T,d)
5     dWo = np.einsum('bti,btj->ij', ctx, dout) # eq: sum_b ctx^T dout
```

```

6   dbo = dout.sum(axis=(0, 1))
7   dctx = dctx.reshape(B, T, H, DK).transpose(0, 2, 1, 3) # (B,H,T,DK)
8   dattn = dctx @ Vh.transpose(0, 1, 3, 2) # gradient w.r.t. attn
9   dVh = attn.transpose(0, 1, 3, 2) @ dctx
10  dscores = softmax_backward(dattn, attn) / np.sqrt(DK)
11  dQh = dscores @ Kh
12  dKh = dscores.transpose(0, 1, 3, 2) @ Qh
13  dQ = dQh.transpose(0, 2, 1, 3).reshape(B, T, d)
14  dK = dKh.transpose(0, 2, 1, 3).reshape(B, T, d)
15  dV = dVh.transpose(0, 2, 1, 3).reshape(B, T, d)
16  dh = dQ @ p['Wq'].T + dK @ p['Wk'].T + dV @ p['Wv'].T
17  dWq = np.einsum('bti,btj->ij', h, dQ) # similarly for Wk, Wv, bias
18  ...
19  return dh, (dWq, dbq, dWk, dbk, dWv, dbv, dWo, dbo)

```

说明:softmax 的反向为 $\delta s = a \odot (\delta a - \langle \delta a, a \rangle)$ (a 为前向 softmax 输出); $\text{einsum}('bti,btj->ij', A, B)$ 等价于 $\sum_{b,t} A[b,t,i]B[b,t,j]$, 即批量转置矩阵乘的求和。

7.6 LayerNorm: 前向与反向

```

1  def layernorm_forward(x, gamma, beta):
2      mu = x.mean(axis=-1, keepdims=True)
3      var = x.var(axis=-1, keepdims=True)
4      std = np.sqrt(var + EPS)
5      xhat = (x - mu) / std
6      return gamma * xhat + beta, (x, mu, var, std, xhat)
7
8  def layernorm_backward(dy, cache, gamma):
9      x, mu, var, std, xhat = cache
10     D = x.shape[-1]
11     dxhat = dy * gamma
12     dvar = (dxhat * (x - mu)).sum(-1, keepdims=True) * (-0.5) * (var+EPS)**(-1.5)
13     dmu = (dxhat * (-1.0/std)).sum(-1, keepdims=True) \
14         + dvar * (-2.0*(x-mu)).sum(-1, keepdims=True) / D
15     dx = dxhat/std + dvar*2.0*(x-mu)/D + dmu/D
16     dgamma = (dy * xhat).sum(axis=(0, 1))
17     dbeta = dy.sum(axis=(0, 1))
18     return dx, dgamma, dbeta

```

说明:反向公式即为第 7 节所述解析梯度,注意 d_{var}, d_{μ} 沿特征维求和($\text{sum}(-1, \text{keepdims=True})$), 而 d_{γ}, d_{β} 沿 batch 与时间维求和。

7.7 前馈网络与整体前向

```

1  def ffn_forward(h, p):
2      f1 = h @ p['W1'] + p['b1']

```

```

3     a = np.maximum(0, f1)                # ReLU
4     return a @ p['W2'] + p['b2'], (h, f1, a)
5
6 def forward(X, p):
7     h = X @ p['We'] + p['be']            # linear embedding (T,4)->(T,32)
8     h = h + PE                           # positional encoding
9     attn_out, c1 = attention_forward(h, p)
10    h1, _ = layernorm_forward(h + attn_out, p['g1'], p['b1ln']) # residual + LN
11    ffn_out, c2 = ffn_forward(h1, p)
12    h2, _ = layernorm_forward(h1 + ffn_out, p['g2'], p['b2ln']) # residual + LN
13    last = h2[:, -1, :]                  # take last position
14    u = last @ p['Wo_final'] + p['b_final']
15    return u, (X, h, c1, c2, h1, h2, last)

```

7.8 数据收集与训练循环

```

1 # data collection: LQR expert + periodic perturbation
2 for ep in range(n_episodes):
3     z = random_init_state(); hist = [z.copy()]
4     for step in range(steps):
5         u = lqr_u(hist[-1])                # expert action
6         if step % 25 == 24:                # periodic perturbation
7             hist[-1][3] += uniform(-1.5, 1.5)
8         z = rk4(hist[-1], u, DT)
9         if abs(z[2]) > pi/4: break
10        hist.append(z)
11        if len(hist) >= T:
12            Xs.append(hist[-T:])            # last T-step states
13            ys.append(lqr_u(hist[-1]))
14
15 # training: MSE + Adam
16 for epoch in range(epochs):
17     for Xb, yb in minibatch(X, y):
18         u, cache = forward(Xb, p)
19         loss = mean((u - yb) ** 2)        # eq(bc)
20         du = 2 * (u - yb) / batch
21         grads = backward(du, cache, p)
22         adam_step(p, grads)

```

7.9 梯度检查

```

1 def grad_check(p, X, y, eps=1e-6):
2     u, cache = forward(X, p)
3     du = 2 * (u - y) / B
4     g = backward(du, cache, p)

```



```

5  for name in p:
6      w = p[name].ravel(); gw = g[name].ravel()
7      for idx in sample_indices:
8          old = w[idx]
9          w[idx] = old + eps; loss_p = loss(forward(X, p), y)
10         w[idx] = old - eps; loss_m = loss(forward(X, p), y)
11         w[idx] = old
12         num = (loss_p - loss_m) / (2 * eps)
13         assert abs(num - gw[idx]) < 1e-4    # numerical vs analytical

```

说明：以中心差分 $(\mathcal{L}(\theta + \epsilon) - \mathcal{L}(\theta - \epsilon))/2\epsilon$ 近似数值梯度，与解析梯度比对，验证反向传播实现正确（实测最大相对误差 $< 10^{-9}$ ）。

8 实验结果

8.1 训练收敛

训练损失由约 0.92 快速降至 10^{-3} 量级。由于 LQR 专家本身是线性的，Transformer 收敛到一个接近线性的策略映射。

8.2 闭环稳定性

闭环测试（800 步 = 8 s，不同初始角度）结果见表 3。

表 3: 闭环稳定测试

初始角度 θ_0	结果	峰值 $ \theta _{\max}$
0.10 rad	稳定	5.7°
0.20 rad	稳定	11.4°
0.30 rad	稳定	17.1°

8.3 扰动鲁棒性

施加 1.2 rad/s 角速度脉冲后，控制器约 3 s 内拉回平衡（峰值 $|\theta| \approx 5.9^\circ$ ）。

8.4 与 LQR 对比与注意力可视化

Transformer 输出与 LQR 高度一致。注意力热力图（4 头 $\times 8 \times 8$ ）中，红框行（当前决策）显示控制器主要关注最近几步状态，符合倒立摆动态的短时记忆特性。

9 手动训练与运行指南

本节给出从零开始手动训练 Transformer 控制器的完整步骤与代码执行顺序，供在本地复现本文结果。

9.1 环境准备

实现仅依赖 Python 3.9+ 与 NumPy (本文使用 Python 3.12、NumPy 2.x) , 无需 PyTorch/TensorFlow 等深度学习框架:

```
1 pip install numpy
```

9.2 文件清单

表 4: 项目文件及其作用

文件	作用
train_transformer_controller.py	单文件自包含训练脚本 (仿真 + LQR + Transformer + 训练 + 导出)
transformer_weights.json	训练产物: 模型权重 (JSON 格式)
transformer_pendulum.html	交互式网页 (内嵌权重, 浏览器直接打开)

9.3 一键运行与脚本内部执行顺序

训练脚本为单文件自包含, 直接运行即可完成全部流程:

```
1 python train_transformer_controller.py
```

脚本按如下顺序执行**六个阶段**, 读者应理解每个阶段的作用与预期输出:

1. **阶段一: LQR 增益求解** (脚本顶层代码, 导入即执行)。构建线性化矩阵 $\mathbf{A}, \mathbf{B}$, 运行离散 Riccati 迭代, 打印反馈增益。预期输出:

```
1 LQR gain K = [ -2.954  -4.735  -46.339  -9.163]
2
```

2. **阶段二: 梯度检查** (函数 `grad_check`)。随机初始化参数, 用中心差分 $(\mathcal{L}(\theta + \epsilon) - \mathcal{L}(\theta - \epsilon)) / 2\epsilon$ 计算数值梯度, 与手写反向传播的解析梯度逐参数比对。预期输出: 各参数相对误差约 $10^{-9} \sim 10^{-7}$, 若出现 10^{-2} 量级误差说明反向传播实现有误, 须先修正再继续。
3. **阶段三: 数据收集** (函数 `collect_data`)。以 LQR 专家驱动仿真, 随机初始状态并周期性扰动, 生成约 5.8×10^4 条 (状态序列, 动作) 样本。预期输出: 打印训练集规模, 如 “58200 samples”。
4. **阶段四: 监督训练** (函数 `train` 内的循环)。初始化 Transformer 参数, 用 Adam 优化 MSE 损失, 训练 60 个 epoch。预期输出: 每 5 个 epoch 打印一次损失, 从约 0.92 下降至 10^{-3} 量级。
5. **阶段五: 闭环测试** (函数 `closed_loop_test`)。将训练好的策略接入闭环, 从 $\theta_0 = 0.1, 0.2, 0.3$ 各跑 800 步, 报告是否失稳与峰值角度。预期输出: 三个初始角度均 “stable”。
6. **阶段六: 导出权重** (函数 `export_weights`)。将全部参数与网络配置序列化为 JSON, 供网页端加载。

9.4 分步手动执行（可选）

若需逐步观察，可将脚本的 `main` 部分注释，逐阶段交互式执行：

```
1 # stage 1: LQR
2 from train_transformer_controller import K_LQR, lqr_u
3 print(K_LQR)
4
5 # stage 2: gradient check
6 from train_transformer_controller import init_params, grad_check
7 grad_check()
8
9 # stage 3: data collection
10 from train_transformer_controller import collect_data
11 X, y = collect_data(300, 200)
12 print(X.shape)          # (N, 8, 4)
13
14 # stage 4: training
15 from train_transformer_controller import train
16 p = train()             # includes grad check + data collection + training
17
18 # stage 5: closed-loop test
19 from train_transformer_controller import closed_loop_test
20 print(closed_loop_test(p, init_th=0.2))
21
22 # stage 6: export
23 from train_transformer_controller import export_weights
24 export_weights(p, 'my_weights.json')
```

9.5 超参数调整

所有可调参数集中在脚本顶部，修改后重跑即可：

- **物理参数**：G, M, m, L（重力、小车质量、杆质量、半长）——改变被控对象；
- **LQR 权重**：`solve_lqr([1.0, 0.1, 50.0, 0.1], 0.1)` 中的 **Q** 对角与 **R**——影响专家行为；
- **Transformer 结构**：D_MODEL, NUM_HEADS, D_FF, T——网络规模与序列长度；
- **训练**：n_episodes, steps_per_episode, epochs, batch, lr——数据量与优化。

9.6 常见问题

- **梯度检查失败**：反向传播实现有误。常见错误为 softmax 沿错维归一化、LayerNorm 的 $d\mu/d\text{var}$ 项遗漏、多头 reshape/transpose 顺序错误。

- **闭环发散**: 检查数据收集是否包含足够扰动; 增大 `n_episodes` 或 θ 初始范围; 或引入 DAgger 迭代。
- **网页无法加载权重**: 确认 `transformer_weights.json` 与 HTML 同目录, 或权重已内嵌进 HTML。

10 结论

本文将倒立摆控制形式化为序列决策任务, 用 Transformer 作为控制器、以 LQR 为专家做模仿学习。实验证明纯 NumPy 手写的 Transformer 控制器能稳定维持平衡并对扰动鲁棒。其优势在于无需精确线性化即可拟合非线性策略、注意力提供可解释性、架构可平滑扩展; 局限在于性能受限于专家与数据分布, 误差累积需 DAgger 修正, 推理计算量高于线性 LQR。

参考文献

- [1] Vaswani, A., et al. (2017). Attention is All You Need. *NeurIPS*, 30.
- [2] Ogata, K. (2010). *Modern Control Engineering* (5th ed.). Prentice Hall.
- [3] Åström, K. J., & Murray, R. M. (2008). *Feedback Systems*. Princeton University Press.
- [4] Kwakernaak, H., & Sivan, R. (1972). *Linear Optimal Control Systems*. Wiley-Interscience.
- [5] Ross, S., Gordon, G., & Bagnell, D. (2011). A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. *AISTATS*.
- [6] Chen, L., et al. (2021). Decision Transformer: Reinforcement Learning via Sequence Modeling. *NeurIPS*.